

1) ABSTRACT

This project develops a comprehensive relational database system for UniHospital, a teaching hospital requiring efficient management of patients, staff, appointments, and medical records. The assignment demonstrates advanced database design and implementation skills, beginning with schema decomposition and normalization to Boyce–Codd Normal Form (BCNF) to ensure data integrity and eliminate redundancy. An Entity–Relationship (ER) model is constructed to capture core entities and relationships, followed by SQL scripts to create tables and seed realistic data. Advanced query techniques, including aggregates, window functions, common table expressions, and subqueries, are applied to extract meaningful insights. Stored procedures, functions, and triggers are implemented to automate business logic and enforce constraints. Indexing strategies and execution plan analysis are used to optimize performance. Transaction management and concurrency control are explored to maintain consistency under multi-user workloads. Finally, administrative tasks such as backup, recovery, and security roles are addressed, ensuring a robust and secure database solution.

2) Introduction and schema design rationale

The effective management of healthcare data is critical to the operation of modern hospitals. UniHospital, a university-affiliated teaching hospital, requires a database system capable of handling diverse information flows, including patient demographics, medical records, staff assignments, and appointment scheduling. This assignment focuses on designing and implementing a relational database that meets these requirements while adhering to best practices in database management. The project begins with schema design and normalization, ensuring that the database structure minimizes redundancy and supports data integrity. An Entity–Relationship (ER) diagram is developed to model the hospital’s core entities and their relationships, forming the foundation for the subsequent SQL implementation. Seed data is generated to simulate realistic hospital operations, enabling meaningful testing of queries and procedures. Advanced SQL queries are employed to demonstrate the system’s analytical capabilities, including aggregate functions, window functions, common table expressions, and subqueries. Programmability features such as stored procedures, user-defined functions, and triggers are integrated to automate workflows and enforce business rules. Performance optimization is achieved through indexing strategies and execution plan analysis, while transaction management and concurrency control ensure reliable multi-user access. Administrative tasks, including backup, recovery, and security role configuration, further strengthen the system’s resilience. By combining theoretical principles with practical implementation, this project illustrates the application of advanced database concepts in a healthcare context. The resulting system not only supports UniHospital’s operational needs but also provides a scalable and secure platform for future expansion.

3) Normalisation proofs (Question 1.1)

Hospital Appointment Schema — Normalisation

Before listing dependencies, a few real-world assumptions need to be made explicit, since the relation doesn't carry a surrogate ApptID:

- A patient has exactly one appointment with a given doctor on a given date, so **{PatientID, DoctorID, ApptDate}** identifies a single appointment/visit.
- Each doctor works in exactly one department (1 doctor $\rightarrow$ 1 department), and each department has one location.
- MedCost is the standard price of a medicine (a property of the drug itself), not something negotiated per prescription.
- Qty is the quantity of a specific medicine prescribed during a specific appointment — this is the only attribute that genuinely needs the full key.

a) Functional Dependencies

F1: PatientID $\rightarrow$ PatientName, PatientPhone

F2: DoctorID $\rightarrow$ DoctorName, Specialisation, DeptName, DeptLocation

F3: DeptName $\rightarrow$ DeptLocation (Unique)

F4: PatientID, DoctorID, ApptDate $\rightarrow$ Diagnosis

F5: MedName $\rightarrow$ MedCost

F6: PatientID, DoctorID, ApptDate, MedName $\rightarrow$ Qty

b) Step-by-step Decomposition

Unnormalised Relation (UNF)

Contains repeating groups (e.g., multiple medications per appointment).

First Normal Form (1NF)

The raw relation has an implicit repeating group: one appointment can have several medicines, each with its own MedName/MedCost/Qty. To reach 1NF, this repeating group is flattened — one row per (appointment, medicine) pair, with all attributes atomic:

AppointmentRecord_1NF (PatientID, PatientName, PatientPhone, DoctorID, DoctorName,
Specialisation, DeptName, DeptLocation, ApptDate,
Diagnosis, MedName, MedCost, Qty)

PK: {PatientID, DoctorID, ApptDate, MedName}

No attribute is dropped — only the repeating group is removed by allowing multiple rows. This relation now satisfies 1NF (atomic attributes, single-valued cells, a defined key)

Second Normal Form (2NF)

2NF requires every non-key attribute to depend on the whole key, not part of it. Checking against the key {PatientID, DoctorID, ApptDate, MedName}:

- ☐ PatientName, PatientPhone depend only on PatientID → partial dependency
- ☐ DoctorName, Specialisation, DeptName, DeptLocation depend only on DoctorID → partial dependency
- ☐ Diagnosis depends only on {PatientID, DoctorID, ApptDate} → partial dependency
- ☐ MedCost depends only on MedName → partial dependency
- ☐ Qty depends on the full key → fully dependent, stays

Removing each partial dependency into its own relation gives:

Patient(PatientID, PatientName, PatientPhone)

(DeptLocation sits in Doctor for now because it is transitively dependent on DoctorID, even though it isn't directly determined by it — 2NF only forbids partial dependency on a subset of the key, not transitive dependency.)

Third Normal Form (3NF)

3NF additionally forbids transitive dependencies — a non-key attribute determined by another non-key attribute. In Doctor, DeptName → DeptLocation is exactly that: DeptLocation depends on DeptName, not directly on the key DoctorID. Splitting this out:

Doctor(DoctorID, DoctorName, Specialisation, DeptName) PK: DoctorID, FK: DeptName
→ Department

Department(DeptName, DeptLocation) PK: DeptName

The other four relations (Patient, Medicine, Appointment, Prescription) each already have only a single non-key attribute or attribute group with no transitive chain, so they're unaffected.

c) Is the final decomposition in BCNF?

BCNF requires that for every non-trivial FD $X \rightarrow Y$ in a relation, X must be a superkey of that relation. Checking each table:

- Patient: only determinant is PatientID, which is the key.
- Doctor: only determinant is DoctorID, which is the key (DeptName doesn't determine anything else here since DeptLocation has been moved out).
- Department: only determinant is DeptName, the key.
- Medicine: only determinant is MedName, the key.
- Appointment: only determinant is {PatientID, DoctorID, ApptDate}, the key.
- Prescription: only determinant is the full key {PatientID, DoctorID, ApptDate, MedName}.

Every relation has exactly one non-trivial determinant, and it coincides with the table's only candidate key — there's no second, overlapping candidate key causing the classic BCNF anomaly.

(d) Lossless-Join & Dependency Preservation

- **Lossless-Join:** Each decomposition preserves the ability to reconstruct the original relation because foreign keys maintain referential integrity (e.g., AppointmentID links Appointment and Prescription).
- **Dependency Preservation:** All original FDs are preserved across relations:
 - PatientID $\rightarrow$ PatientName, PatientPhone (in Patient)
 - DoctorID $\rightarrow$ DoctorName, Specialisation, DeptID (in Doctor)
 - DeptID $\rightarrow$ DeptName, DeptLocation (in Department)
 - MedID $\rightarrow$ MedName, MedCost (in Medication)
 - AppointmentID $\rightarrow$ PatientID, DoctorID, ApptDate, Diagnosis (in Appointment)
 - (AppointmentID, MedID) $\rightarrow$ Qty (in Prescription)

Thus, both conditions are satisfied.

4) ER diagram

1. Entities and Attributes

Primary keys are **bolded**, foreign keys are *italicized*.

- **Department:** **DeptID**, DeptName, Location, *HeadDoctorID*
- **Doctor:** **DoctorID**, FirstName, LastName, Specialisation, Phone, HireDate, *DeptID*
- **Ward:** **WardID**, WardName, Capacity, *DeptID*
- **Patient:** **PatientID**, FirstName, LastName, DOB, Gender, Address, Phone, InsuranceNo
- **Appointment:** **AppointmentID**, *PatientID*, *DoctorID*, ApptDate, ApptTime, Status, Notes
- **Admission:** **AdmissionID**, *PatientID*, *WardID*, AdmitDate, DischargeDate, DiagnosisCode
- **Medication:** **MedID**, MedName, DosageForm, UnitCost, StockQty
- **Prescription:** **PrescID**, *AdmissionID*, *MedID*, *DoctorID*, Quantity, PrescDate
- **Bill:** **BillID**, *PatientID*, *AdmissionID*, TotalAmount, PaidAmount, BillDate, Status
- **Staff:** **StaffID**, FirstName, LastName, Role, *DeptID*, Salary, StartDate

2. Relationship Cardinality & Participation Matrix

The structural rules derived directly from your constraints indicate how entities interact:

Relationship Name	Entity A	Cardinality & Participation (A)	Entity B	Cardinality & Participation (B)	Constraint Rule Used
Works In	Department	1 and Only One (Total)	Doctor	Zero or Many (Partial)	Doctor.DeptID INT NOT NULL
Heads	Doctor	Zero or One (Partial)	Department	Zero or One (Partial)	Department.HeadDoctorID INT NULL
Belongs To	Department	1 and Only One (Total)	Ward	Zero or Many (Partial)	Ward.DeptID INT NOT NULL
Employs	Department	1 and Only One (Total)	Staff	Zero or Many (Partial)	Staff.DeptID INT NOT NULL

Schedules	Patient	1 and Only One (Total)	Appointment	Zero or Many (Partial)	Appointment.PatientID INT NOT NULL
Attends	Doctor	1 and Only One (Total)	Appointment	Zero or Many (Partial)	Appointment.DoctorID INT NOT NULL
Undergoes	Patient	1 and Only One (Total)	Admission	Zero or Many (Partial)	Admission.PatientID INT NOT NULL
Hosts	Ward	1 and Only One (Total)	Admission	Zero or Many (Partial)	Admission.WardID INT NOT NULL
Receives	Admission	1 and Only One (Total)	Prescription	Zero or Many (Partial)	Prescription.AdmissionID INT NOT NULL
Contains	Medication	1 and Only One (Total)	Prescription	Zero or Many (Partial)	Prescription.MedID INT NOT NULL
Writes	Doctor	1 and Only One (Total)	Prescription	Zero or Many (Partial)	Prescription.DoctorID INT NOT NULL
Issued To	Patient	1 and Only One (Total)	Bill	Zero or Many (Partial)	Bill.PatientID INT NOT NULL
Generated For	Admission	Zero or One (Partial)	Bill	Zero or Many (Partial)	Bill.AdmissionID INT NOT NULL

3. Participation constraints (total / partial).

[Staff]

| (Many-to-One, Total)

v

[Department] <=====(One-to-One, Partial Head)=====> [Doctor]

|

| (One-to-Many, Total)

v

[Ward]

|

| (One-to-Many, Total)

v

[Admission] <=====(One-to-Many, Total)=====+====> [Patient]

| ^

| +------(Many-to-One, Partial)-----+ | (One-to-Many, Total)

| | v

| (One-to-Many, Total) +----> [Bill]

v

[Prescription] <=====(Many-to-One, Total)=====> [Medication]

5) Query design decisions and results

1. Aggregate Queries

Decision: Aggregates (COUNT, SUM, AVG, MIN, MAX) were chosen to provide statistical insights into hospital operations.

Example:

```
SELECT DeptID, COUNT(*) AS DoctorCount
FROM Doctor
GROUP BY DeptID;
```

Result: Returns the number of doctors per department, supporting resource allocation analysis.

2. Window Functions

Decision: Window functions (ROW_NUMBER, RANK, NTILE) were applied to analyze patient visit frequency and appointment scheduling trends.

Example:

```
SELECT PatientID, ApptDate,
       ROW_NUMBER() OVER (PARTITION BY PatientID ORDER BY ApptDate) AS
VisitOrder
FROM Appointment;
```

Result: Produces sequential visit numbers per patient, useful for tracking patient history.

3. Common Table Expressions (CTEs)

Decision: CTEs were used to simplify complex queries, particularly for hierarchical relationships and temporary result sets.

Example:

```
WITH FrequentPatients AS (
  SELECT PatientID, COUNT(*) AS VisitCount
  FROM Appointment
  GROUP BY PatientID
  HAVING COUNT(*) > 5
)
SELECT p.PatientName, f.VisitCount
FROM Patient p
JOIN FrequentPatients f ON p.PatientID = f.PatientID;
```

Result: Identifies patients with more than five appointments, aiding in chronic case management.

4. Subqueries

Decision: Subqueries were employed to filter results based on dynamic conditions, such as identifying doctors with above-average patient loads.

Example:

```
SELECT DoctorID, DoctorName
FROM Doctor
WHERE DoctorID IN (
    SELECT DoctorID
    FROM Appointment
    GROUP BY DoctorID
    HAVING COUNT(*) > (
        SELECT AVG(AppCount)
        FROM (
            SELECT COUNT(*) AS AppCount
            FROM Appointment
            GROUP BY DoctorID
        ) AS AvgLoad
    )
);
```

Result: Returns doctors whose appointment counts exceed the average, supporting workload balancing.

5. Query Validation

- **Correctness:** Queries were tested against seeded data to ensure accurate outputs.
- **Performance:** Execution plans were reviewed to confirm efficiency.
- **Relevance:** Each query aligns with hospital management needs (resource allocation, patient tracking, workload distribution).

6) Index analysis: before/after statistics

Execution Plan Analysis Answers

a) IX_Bill_Unpaid — unpaid balance filter

STATISTICS IO / TIME (expected)

Metric	Without index	With index
Logical reads	Table scan — all pages	2–3 index leaf pages
CPU time	~15 ms (full scan)	~1 ms
Elapsed time	~20 ms	~2 ms

Estimated vs actual rows

Without the index SQL Server has no statistics on `TotalAmount > PaidAmount`, so it estimates ~30% of rows (~4 rows from 15). Actual rows returned depend on real data — with this dataset, 14 of 15 bills have an outstanding balance. After index creation, SQL Server builds column statistics and its estimate narrows significantly.

Plan operators to watch

Operator	Appears when?	How to eliminate
<i>Table Scan</i>	No index, small table; optimizer skips indexes under ~100 rows	Index only helps once the table grows. On a dev dataset SQL Server may still table scan — this is correct for small sets.
<i>Key Lookup</i>	Index chosen but SELECT * needed columns not in INCLUDE list	Already mitigated — <code>TotalAmount</code> , <code>PaidAmount</code> , <code>BillDate</code> are all in INCLUDE. Use column-specific SELECT instead of *.

On the 15-row dev dataset the optimizer will likely choose a table scan regardless of the index. Test on a larger dataset (10k+ rows) to see the index kick in. Use `WITH (INDEX(IX_Bill_Unpaid))` to force it for plan comparison.

b) IX_Admission_Ward_Active — active admissions by ward

STATISTICS IO / TIME (expected)

Metric	Without index	With index
Logical reads	Full scan (15 rows, ~3 pages)	1–2 index seeks
CPU time	~8 ms	<1 ms

Estimated vs actual rows

SQL Server estimates selectivity from the two key columns. `WardID = X` reduces to roughly 1/5 of rows (3 from 15). `DischargeDate IS NULL` is trickier — SQL Server uses a default 30% estimate for IS NULL when no histogram exists. In this dataset all 15 admissions have a discharge date set, so actual = 0, while estimated may be ~1–2. This estimated-vs-actual gap is common.

Plan operators to watch

Operator	Appears when?	How to eliminate
<i>Index Scan</i>	Index exists but SQL Server reads all entries (e.g., when >30% of rows match or stats are stale)	Run <code>UPDATE STATISTICS Admission</code> after loading data. Filtered index can help: add <code>WHERE DischargeDate IS NULL</code> to the index definition.

<i>Key Lookup</i>	If AdmitDate or DiagnosisCode needed but not in INCLUDE	Already covered by INCLUDE list.
-------------------	---	----------------------------------

Consider a *filtered* index: CREATE NONCLUSTERED INDEX IX_Admission_Ward_Active ON Admission (WardID) INCLUDE (PatientID, AdmitDate, DiagnosisCode) WHERE DischargeDate IS NULL — this is smaller, faster, and makes the IS NULL predicate trivially satisfied.

c) IX_Prescription_Admission_Date — prescriptions by admission

STATISTICS IO / TIME (expected)

Metric	Without index	With index
Logical reads	Full scan (20 rows)	Index seek + 1–2 leaf reads
Sort operator	Explicit Sort on PrescDate	Eliminated — index already ordered

Estimated vs actual rows

With AdmissionID as the leading key column, SQL Server seeks directly to the matching range. Estimated $\approx 20/\text{distinct_admissions}$; actual = however many prescriptions that admission has. The sort is free because PrescDate is the second key column — the rows come out in order from the index leaf pages.

Plan operators to watch

Operator	Appears when?	How to eliminate
<i>Sort</i>	ORDER BY column not in index key	Already prevented — PrescDate is in the key (not just INCLUDE), guaranteeing ordered retrieval.
<i>Key Lookup</i>	SELECT * pulls columns outside INCLUDE	Replace SELECT * with explicit column list matching the INCLUDE columns.

This is the cleanest index of the four — leading equality key (AdmissionID) followed by a range/order key (PrescDate) is a textbook composite index pattern.

d) Full-text index on Appointment.Notes

STATISTICS IO / TIME (expected)

Metric	Without FT index (LIKE)	With CONTAINS()
Logical reads	Full table scan every time	FT index lookup + base table fetch
CPU time	Grows linearly with row count	Sub-linear; inverted index lookup

Estimated vs actual rows

CONTAINS() does not flow through the standard cardinality estimator the same way B-tree predicates do. SQL Server shows a fixed estimate (often 1 row) for full-text predicates regardless of actual matches. Expect a large estimated-vs-actual gap — this is expected and not a plan quality problem.

Plan operators to watch

Operator	Appears when?	How to eliminate
<i>Full-Text Match + Hash Join</i>	Always present with FT search — the FT engine returns RID/key matches, then joins to base table	Cannot be eliminated. Normal behavior. Reduce columns in SELECT to minimise Key Lookup cost after the join.
<i>Index Scan on base table</i>	If FT selectivity estimate is high, optimizer may skip the FT path	Force FT path with OPTION (USE HINT ('ENABLE_PARALLEL_PLAN_PREFERENCE')) or restructure as a CTE pre-filtering with CONTAINS.

Capture execution plans with: SET STATISTICS IO ON; SET STATISTICS TIME ON; before the query, then use SET STATISTICS IO OFF; SET STATISTICS TIME OFF; after. For graphical plans use SET ACTUAL EXECUTION PLAN ON or Ctrl+M in SSMS.

e) logical reads and elapsed time

1. STATISTICS IO and TIME

- **Before columnstore index:**
 - Expect **high logical reads** (full table scan).
 - CPU time and elapsed time will be higher.
- **After columnstore index:**
 - Logical reads drop significantly.
 - CPU time improves due to compressed columnar storage.
 - Elapsed time reduced because aggregation is vectorized.

2. Execution Plan Analysis

- **Before index:**
 - Likely shows a **Clustered Index Scan** or **Heap Scan** on Bill.

- Aggregations performed row-by-row.
- **After index:**
 - Plan switches to a **Columnstore Index Scan**.
 - Aggregations are batch-mode (vectorized), reducing CPU cost.
 - Estimated vs. Actual rows align more closely because columnstore maintains better statistics.

3. Operators Eliminated

- **Key Lookup:** Not needed because columnstore index covers all columns used in the query.
- **Hash Join / Hash Aggregate:** Often replaced by **Batch Mode Aggregate**, which is faster.
- **Index Scan:** Replaced by **Columnstore Scan**, which is more efficient for large datasets.

7) Transaction and concurrency discussion

a) Resolving the Deadlock

Two strategies are required:

1. Consistent Lock Order

- Always acquire locks in the same sequence (e.g., Patient first, then Bill).
- This prevents circular waits because transactions won't block each other in opposite order.

2. READ_COMMITTED_SNAPSHOT Isolation

- Enables **row versioning** instead of blocking reads.
- Readers don't block writers, and writers don't block readers.
- This reduces contention and deadlock probability in workloads with frequent reads and updates.

b) Choosing SNAPSHOT vs. SERIALIZABLE

• SNAPSHOT Isolation

- Provides a consistent view of data at the start of the transaction using row versions.
- Allows high concurrency: multiple users can read without blocking each other.
- Best for hospital reporting queries (e.g., billing summaries, patient histories) where you need consistency but don't want to block ongoing updates.

- **SERIALIZABLE Isolation**

- The strictest level: transactions behave as if executed one after another.
- Prevents phantom reads, but at the cost of heavy locking and blocking.
- Suitable for **critical operations** like assigning beds or ensuring no duplicate patient admissions, where correctness outweighs performance.

In a hospital management context, you'd choose **SNAPSHOT** for analytical/reporting workloads to maximize concurrency, and **SERIALIZABLE** for transactional integrity in core clinical processes.

Bonus C — Performance Tuning Dashboard

Create a T-SQL reporting script that produces a real-time dashboard covering:

1. Top-10 Most Expensive Queries

- Source: sys.dm_exec_query_stats
- These DMVs track CPU time, logical reads, and execution counts.

Example snippet: This shows queries that are slowest on average.

```
SELECT TOP 10
    qs.total_elapsed_time / qs.execution_count AS AvgElapsedTime,
    qs.execution_count,
    SUBSTRING(st.text, 1, 200) AS QueryText
FROM sys.dm_exec_query_stats qs
CROSS APPLY sys.dm_exec_sql_text(qs.sql_handle) st
ORDER BY AvgElapsedTime DESC;
```

2. Missing Index Recommendations

- Source: sys.dm_db_missing_index_details
- SQL Server suggests indexes based on workload.

Example: The script below decide where to add nonclustered indexes.

```
SELECT
    mid.database_id, mid.[object_id],
    mid.equality_columns, mid.inequality_columns,
    mid.included_columns
FROM sys.dm_db_missing_index_details mid;
```

3. Wait Statistics by Wait Type

- Source: sys.dm_os_wait_stats
- Shows bottlenecks like I/O waits, lock waits, or CPU scheduling.

Example: This below Identifies systemic issues

```
SELECT TOP 20
    wait_type, waiting_tasks_count, wait_time_ms
FROM sys.dm_os_wait_stats
ORDER BY wait_time_ms DESC;
```

4. Current Blocking Chains

- Source: sys.dm_exec_requests
- Reveals which sessions are blocking others.

Example: Lets you trace chains of blocking queries.

```
SELECT
    blocking_session_id, session_id, wait_type, wait_time, wait_resource
FROM sys.dm_exec_requests
WHERE blocking_session_id <> 0;
```

8. Administration and security approach

The database security strategy follows the principle of least privilege. Separate database roles were created for clinicians, billing staff, and read-only users. Each role receives only the permissions necessary to perform assigned duties. This reduces the likelihood of accidental data modification and improves compliance with healthcare data protection requirements. Backup and recovery procedures include full, differential, and transaction log backups. This approach supports point-in-time recovery and minimizes potential data loss in the event of hardware failure or user error. Monitoring scripts are used to identify index fragmentation and outdated statistics, ensuring continued query performance.

9) Conclusion and reflections

This project has demonstrated the application of advanced database management principles within the context of UniHospital, a teaching hospital requiring robust information systems. Beginning with schema design and normalization to Boyce–Codd Normal Form, the work ensured that redundancy was eliminated and data integrity preserved. The Entity–Relationship model provided a clear representation of hospital operations, forming the foundation for SQL implementation. Through the use of advanced queries — including aggregates, window functions, common table expressions, and subqueries — the database was shown to support complex analytical needs. Programmability features such as stored procedures, functions, and triggers automated workflows and enforced business rules. Performance optimization was achieved through indexing strategies and execution plan analysis, while transaction management and concurrency control safeguarded consistency under multi-user workloads. Administrative tasks, including backup, recovery, and security role configuration, further strengthened the resilience of the system.

Reflecting on the process, the assignment highlighted the importance of balancing theoretical rigor with practical implementation. Normalization proofs and ER modeling reinforced the value of structured design, while query optimization and transaction handling emphasized the realities of performance and reliability in production environments. The integration of administrative tasks underscored the holistic nature of database management, extending beyond design and queries to encompass long-term sustainability and security. Overall, the project illustrates how advanced database concepts can be applied to solve real-world challenges in healthcare, providing a scalable and secure platform for future expansion.

10. References

1. Codd, E.F. (1970) *A relational model of data for large shared data banks*. Communications of the ACM, 13(6), pp.377–387.
2. Elmasri, R. and Navathe, S.B. (2016) *Fundamentals of Database Systems*. 7th edn. Boston: Pearson.
3. Ramakrishnan, R. and Gehrke, J. (2003) *Database Management Systems*. 3rd edn. New York: McGraw-Hill.
4. Microsoft (2024). Transact-SQL Reference (Database Engine). <https://learn.microsoft.com/en-us/sql/t-sql/>
5. Ramakrishnan, R. & Gehrke, J. (2003). Database Management Systems (3rd ed.). McGraw-Hill. Chapters 12–16.
6. Elmasri, R. & Navathe, S. (2015). Fundamentals of Database Systems (7th ed.). Pearson. Chapters 14–17.
7. Forta, B. (2020). SQL in 10 Minutes a Day (5th ed.). Sams.
8. Brust, A. J. & Forte, S. (2012). Professional Microsoft SQL Server 2012 Programming. Wrox.